

BANGLACARE

Entity-Aware Multi-Task Bangla Health Query Triage and Medical NER

Complete Start-to-Finish Implementation Guide - One Best Version

Final outputs	Medical entities + 4-class severity + calibrated confidence
Main encoder	2-layer BiLSTM, shared by NER and severity
Representations	Pretrained Bengali FastText 300D + custom Medical FastText 300D
Target platform	Kaggle Free GPU / standard Python NLP stack

No code is included. This document fixes the methodology and explains exactly what to build, in what order, and what each phase should produce.

Prepared: September 2026

How to use this guide

Follow the phases in order. Do not start the main neural network before the dataset splits, leakage checks, medical corpus, and both FastText resources are ready. Each phase ends with a concrete output or checkpoint. If that output is not correct, fix it before moving forward.

Frozen design

This guide intentionally gives one fixed custom-model version. It does not include a Character-CNN, hand-made entity-count features, class-weighted severity loss, or a transformer inside the final model. Those were considered earlier but are excluded from the final version to keep the system justified, coherent, and efficient.

Table of contents

1. Project definition and final input/output
 2. Resources and downloads
 3. Phase 1 - Workspace and raw-data audit
 4. Phase 2 - Splitting and leakage prevention
 5. Phase 3 - Text preprocessing and tokenization
 6. Phase 4 - Medical corpus construction
 7. Phase 5 - Train the custom Medical FastText
 8. Phase 6 - Prepare pretrained Bengali FastText
 9. Phase 7 - Build model-ready token representations
 10. Final neural architecture
 11. Phase 8 - NER pretraining
 12. Phase 9 - Severity-head warm-up
 13. Phase 10 - Joint multi-task fine-tuning
 14. Phase 11 - Checkpoint selection and confidence calibration
 15. Phase 12 - Final evaluation
 16. Required ablations and baseline
 17. Final inference and demo
 18. Project files and outputs
 19. Kaggle runtime plan
 20. Common failure checks
 21. Final report structure
 22. Start-to-finish checklist
- References and download links

1. Project definition and final input/output

The project builds one lightweight multi-task NLP system for informal Bangla healthcare queries. A user enters one health-related message. The same shared neural encoder produces two outputs: medical named entities and a four-class severity decision. The system is a routing/triage aid, not a diagnostic or treatment system.

1.1 Input

One free-form health query. The model should tolerate Bangla, Banglish/Romanized Bangla, English medical terms, code-switching, numbers, medicine names, dosage expressions, spelling variation, and informal writing.

Example input:

আমার বয়স ২৭ বছর। হঠাৎ বুকে অনেক ব্যথা হচ্ছে এবং শ্বাস নিতে কষ্ট হচ্ছে। **Napa** খেয়েছি।

1.2 Output A - Medical named entities

The NER branch extracts seven medical entity categories used by Bangla-HealthNER [2]: Symptom, Health Condition, Medicine, Age, Dosage, Specialist, and Medical Procedure. Internally the model uses IOB/BIO-style sequence labels: B-type, I-type, and O. Seven entity types therefore produce 15 token labels in total.

Entity type	Example meaning
Symptom	pain, fever, breathing difficulty
Health Condition	disease/condition or health event
Medicine	medicine or drug name
Age	patient age expression
Dosage	dose/frequency expression
Specialist	medical specialist
Medical Procedure	test, surgery, procedure

1.3 Output B - Severity

The severity branch predicts exactly one of four labels from the Bangla Healthcare Severity Dataset [1]: Emergency, Urgent, Routine, or General Query. Your audit shows all four classes are approximately 22% to 27% of the data, so the dataset is already close to balanced.

Training consequence

Use ordinary multi-class cross-entropy for severity. Do not use class weights, SMOTE, oversampling, or undersampling in the main version. Keep the split stratified and still report per-class recall, especially Emergency Recall.

1.4 Output C - Confidence

The severity head produces four logits/probabilities. After the final model is selected, calibrate the probabilities on the validation set using temperature scaling. The displayed confidence should come from the calibrated probability, not raw softmax confidence.

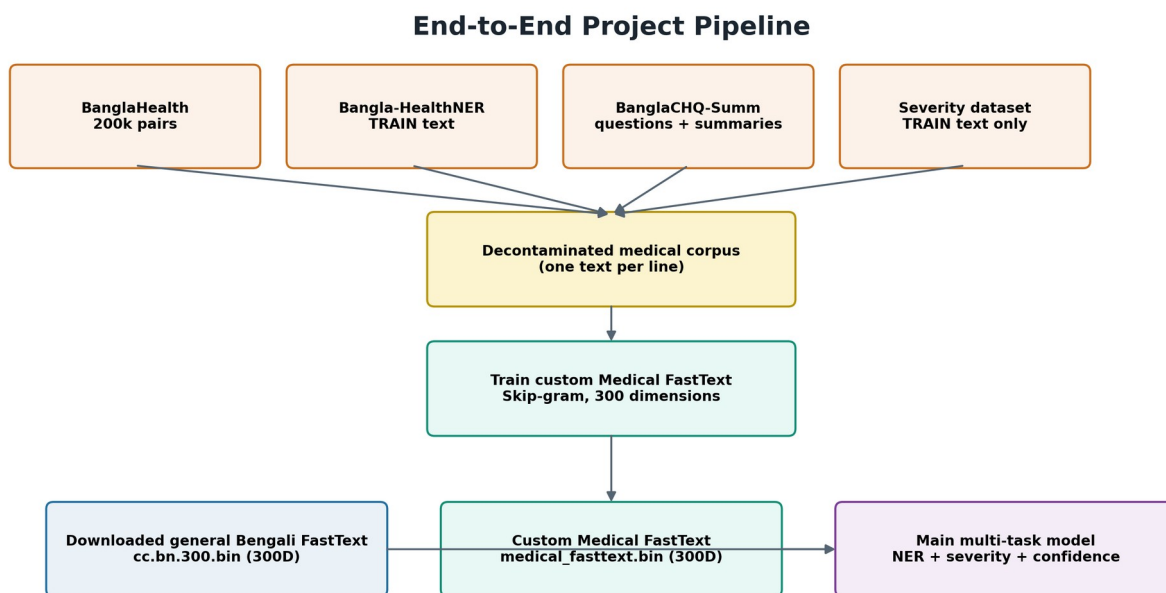
1.5 Example final response

Entities:

Symptom -> বৃকে অনেক ব্যথা
 Symptom -> শ্বাস নিতে কষ্ট
 Medicine -> Napa
 Age -> ২৭ বছর

Severity: Emergency
 Confidence: 0.87

The application should never add a diagnosis such as “heart attack” unless diagnosis is separately designed, validated, and clinically governed. Diagnosis is outside this project.



Validation/test texts are excluded from custom corpus training. “Action Needed” is never used as a model input.

Figure 1. Complete project pipeline.

2. Resources and downloads

You already have the first three supervised/in-domain resources. The only additional external data/model resources required by this final version are BanglaHealth and pretrained Bengali FastText.

Resource	Role	Status
Bangla-HealthNER	Supervised NER labels + in-domain training text	Already downloaded
Bangla Healthcare Severity Dataset	Four-class severity supervision	Already downloaded
BanglaCHQ-Summ	Additional consumer-health text for corpus	Already downloaded
BanglaHealth	Large health-domain unlabeled/paraphrase corpus	Load from Hugging Face
Bengali FastText cc.bn.300.bin	General pretrained Bengali subword embedding	Download once

2.1 Exact links

Bangla-HealthNER - Dataset/repository for seven-type medical NER

<https://github.com/alvi-khan/Bangla-HealthNER>

Bangla Healthcare Severity Dataset - 5,263 manually annotated queries; target column is Categories

<https://data.mendeley.com/datasets/37z8kgx79m/1>

BanglaCHQ-Summ - 2,350 consumer-health question-summary pairs

<https://github.com/alvi-khan/BanglaCHQ-Summ>

BanglaHealth - 200,000 health-domain source/paraphrase pairs; load using Hugging Face Datasets

<https://huggingface.co/datasets/faisal4590aziz/bangla-health-related-paraphrased-dataset>

Pretrained Bengali FastText - Official 300D Common Crawl + Wikipedia Bengali model

<https://fasttext.cc/docs/en/crawl-vectors.html>

Direct FastText binary - Compressed Bengali binary model; extract to cc.bn.300.bin

<https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.bn.300.bin.gz>

2.2 BanglaHealth handling

Do not manually download BanglaHealth unless you want a permanent local copy. In Kaggle, load it through the Hugging Face Datasets library. The dataset has 200,000 rows, each with a source sentence and a paraphrased sentence [4]. Both text columns can be used as unlabeled corpus material. Use BanglaHealth only for representation learning; do not create severity or NER labels from it.

2.3 What you do NOT need

- BanglaBioMed is not required for the final version.
- Word2Vec and GloVe are not required.
- A Character-CNN is not used because FastText already supplies character/subword information.
- BanglaBERT/BanglishBERT are not part of this fixed custom-model implementation.
- No extra medical ontology or knowledge graph is required.

3. Phase 1 - Workspace and raw-data audit

Before cleaning anything, preserve untouched copies of every source. Reproducibility becomes much easier if raw files are never overwritten.

3.1 Recommended top-level folders

```
banglacare/
  data/raw/
  data/splits/
  data/processed/
  corpus/
  embeddings/
  checkpoints/
  results/
  figures/
  app/
  logs/
```

3.2 Audit each dataset before splitting

- Record the number of rows/samples and column names.
- Check missing or empty texts.
- Check exact duplicate texts after only Unicode/whitespace normalization.
- Inspect text-length distribution: median, 90th, 95th, 99th percentile, and maximum token length.
- Inspect Bangla, Latin/Banglish, mixed-script, digits, medicine names, URLs, and unusual characters.
- For severity data, verify the four class counts. Your current inspection shows every class lies around 22%-27%.
- For HealthNER, verify every sample has the same number of tokens and labels. Do not silently repair mismatches without inspection.
- For BanglaCHQ-Summ, identify the exact question and summary text fields.
- For BanglaHealth, verify the source_sentence and paraphrased_sentence columns before corpus construction.

3.3 Severity dataset leakage rule

Critical

Use Text as input and Categories as the target. Do not use Action Needed as an input feature, auxiliary target, prompt, corpus sentence, or derived rule. It is annotation-related information and can leak the answer [1].

3.4 Save an audit report

Create one small CSV/JSON/text report containing row counts, missing values, duplicate counts, class percentages, length percentiles, and label sets. This becomes evidence that you understood the data before modeling.

Phase output: raw data preserved + dataset_audit report.

4. Phase 2 - Splitting and leakage prevention

4.1 HealthNER split

Use the official train/validation/test files supplied by Bangla-HealthNER. Do not reshuffle them. NER validation is used for early stopping and model selection; NER test is touched only after the final model is frozen.

4.2 Severity split

Create a stratified 80% train / 10% validation / 10% test split. Because there are 5,263 samples, this is roughly 4.2k training queries and about 0.5k each for validation and test, but exact counts can change slightly when duplicate groups are kept together.

The four severity classes are already close to balanced (22%-27%), so stratification is enough; do not rebalance the data.

4.3 Duplicate-aware grouping BEFORE severity split

Exact or nearly identical questions must not be allowed to appear in different splits. Otherwise the model may memorize wording and produce unrealistically high test results.

1. Normalize text only for duplicate detection (Unicode normalization + whitespace cleanup + lowercase Latin).
2. Group exact duplicates.
3. For suspicious near-duplicates, compare character n-gram similarity. A high threshold such as about 0.90 Jaccard/cosine similarity is a practical flag for manual/grouped review, not a universal truth.
4. Assign the whole duplicate/near-duplicate group to one split.
5. Then preserve the four-class distribution as closely as possible.

4.4 Create a held-out blacklist

After splits are frozen, create a list/fingerprint set containing every HealthNER validation/test text and every severity validation/test text. This is used only to prevent those texts from entering your custom medical corpus.

HELD_OUT = HealthNER(val + test) + Severity(val + test)

These texts are never used to train the custom Medical FastText.

4.5 Why this matters

FastText training is unsupervised, but allowing test sentences into the custom embedding corpus still gives the model advance exposure to test wording. Preventing this makes the evaluation cleaner and easier to defend.

Phase output: frozen HealthNER splits + duplicate-aware severity train/val/test + held-out fingerprint list.

5. Phase 3 - Text preprocessing and tokenization

Medical text should be cleaned conservatively. Aggressive “normalization” can destroy the exact clues the model needs.

5.1 Keep these

- Negation words such as না, নেই, হয়নি.
- Bangla and Arabic digits: ২৭, 27, 500.
- Dose patterns: 500mg, 0+1+1, 2 times/day.
- Medicine and brand names.
- English and Romanized Bangla words.
- Punctuation that can carry meaning.
- Original spelling variation unless it is clearly a Unicode/whitespace artifact.

5.2 Clean these

- Normalize Unicode consistently (NFC is a sensible default).
- Collapse repeated/irregular whitespace.
- Remove BOM and stray zero-width spaces; do not blindly delete legitimate script joiners.
- Lowercase Latin text consistently if you choose lowercase as the project convention.
- Handle URLs/emails consistently (remove or replace with placeholders if they are not task signals).
- Remove truly empty or corrupted rows after logging them.

5.3 Do NOT do these

- Do not remove stopwords.
- Do not stem or lemmatize Bangla aggressively.
- Do not translate Banglish into Bangla.
- Do not translate English medical words.
- Do not remove numbers.
- Do not replace medicine names with generic tokens.
- Do not use severity labels or Action Needed during text cleaning.

5.4 Tokenization rule

Use the tokenization that preserves HealthNER label alignment. For HealthNER, treat the provided token sequence/whitespace segmentation as authoritative. For severity and new queries, use the same simple whitespace-oriented tokenization after conservative cleanup. This keeps the shared model consistent and avoids breaking IOB labels.

Sanity gate

For every HealthNER example: number of tokens must equal number of NER labels. If this fails, stop and fix preprocessing before any model training.

5.5 Sequence length

Use dynamic padding inside each batch. After the audit, choose a maximum length that covers almost all training examples; a cap of 512 tokens is a practical upper bound for this BiLSTM on a T4. If only a tiny tail exceeds 512, log and truncate those cases. If a substantial fraction exceeds 512, use overlapping 512-token chunks rather than silently discarding large parts of the text.

Phase output: deterministic preprocessing/tokenization rules + cleaned split files with unchanged labels.

6. Phase 4 - Medical corpus construction

The custom Medical FastText should see a large health-domain corpus, but it must never see your held-out task texts. The corpus is unlabeled: one normalized text sequence per line.

6.1 Include exactly these sources

Source	Text included	Reason
BanglaHealth	source_sentence + paraphrased_sentence	Large health-domain vocabulary and paraphrases
Bangla-HealthNER	TRAIN text only	Informal medical questions/answers and entity vocabulary
BanglaCHQ-Summ	questions + summaries	Consumer-health language
Severity dataset	TRAIN Text only	Exact target-domain style

6.2 Exclude exactly these

- HealthNER validation text.
- HealthNER test text.
- Severity validation text.
- Severity test text.
- Severity Action Needed column.
- Any labels or annotation strings.
- Exact or near-duplicate corpus lines that match held-out texts.

6.3 Deduplication order

6. Normalize candidate corpus lines using the same light text normalization.
7. Remove empty lines.
8. Remove exact duplicate lines.
9. Check candidate lines against the held-out fingerprint list; remove matches.
10. Run near-duplicate checking specifically against held-out texts and remove suspicious high-similarity matches.
11. Write the final corpus with one text per line.

6.4 Important note about BanglaHealth

BanglaHealth contains machine-generated paraphrases and news-derived health sentences [4]. This is fine for unsupervised embedding learning, but do not treat its paraphrases as gold medical labels. HealthNER and the severity dataset remain the supervised truth sources.

Phase output: corpus/medical_corpus.txt + corpus statistics (line count, unique-token count, script mix).

7. Phase 5 - Train the custom Medical FastText

FastText is a shallow word/subword embedding model, not your final classifier. It learns a vector for each word from surrounding context and also learns character n-gram components, which helps rare words, spelling variants, Banglish, and medicine names [5].

7.1 Fixed training configuration

Setting	Final value	Meaning
Training objective	Skip-gram	Predict surrounding words from the current word
Dimension	300	300 numbers per medical word vector
Context window	5	Use nearby words within a small window
Epochs	15	Pass through the corpus 15 times
minCount	2	Ignore one-off vocabulary items as standalone words
Character n-grams	minn=3, maxn=6	Build word vectors from subword pieces
Negative samples	10	Contrast true contexts with negative samples
Learning rate	0.05	Standard starting rate for this FastText run

7.2 What the model should learn

General health relationships plus domain-specific contexts: symptoms, drug names, procedures, Banglish spellings, informal phrases, and medically meaningful morphology. The goal is not to classify severity at this stage.

7.3 Validate the embedding qualitatively

Before using it in the main model, inspect nearest neighbors for a small audit set: common symptoms, medicine names, procedure terms, Banglish spellings, and rare medical words. You are checking that the model behaves like a medical embedding rather than random vectors.

Do not expect every neighbor to be medically perfect. FastText learns usage similarity from the corpus, not an ontology.

7.4 Save

```
embeddings/medical_fasttext.bin
embeddings/medical_fasttext.vec (optional text export)
logs/medical_fasttext_config.json
results/embedding_neighbor_audit.txt
```

Phase output: a reusable 300D medical FastText binary model.

8. Phase 6 - Prepare pretrained Bengali FastText

Download the official Bengali Common Crawl + Wikipedia FastText binary. The official crawl vectors are 300-dimensional and use FastText character n-grams [5]. Use the binary .bin form because it can construct vectors for out-of-vocabulary strings from subwords.

Direct file - Download and extract this file to obtain cc.bn.300.bin

<https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.bn.300.bin.gz>

8.1 Why keep both FastText models

The general Bengali model contributes broad language knowledge from a very large corpus. Your Medical FastText contributes domain/context knowledge from health text, including informal and task-specific vocabulary. We do not try to merge the two FastText models beforehand. We simply query both and concatenate their vectors for each token.

8.2 Keep both embedding models fixed

During BiLSTM training, treat the two FastText models as fixed feature generators. The trainable projection layer learns how to combine them. This reduces overfitting and prevents a small supervised dataset from destroying the structure learned by the embeddings.

Phase output: embeddings/cc.bn.300.bin + embeddings/medical_fasttext.bin.

9. Phase 7 - Build model-ready token representations

When a query is fed to the main model, each token is converted into two vectors.

$$\mathbf{x}_i = [\text{GeneralFastText}(w_i) ; \text{MedicalFastText}(w_i)]$$

$$300D + 300D = 600D \text{ per token.}$$

FastText can construct vectors for unseen spellings from subwords, so an unseen medical/Banglish token does not have to collapse to a single generic UNK vector.

9.1 Projection

$$600D \rightarrow \text{Linear} \rightarrow 256D \rightarrow \text{LayerNorm} \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.20)$$

This is trainable and learns how much of each embedding source matters.

The projected 256D sequence, not the raw 600D sequence, is fed into the BiLSTM. This keeps the recurrent network compact.

9.2 Efficient implementation idea

FastText lookup itself does not need GPU training. Cache token vectors so the same token is not recomputed repeatedly. The FastText binaries remain available for genuinely new words at inference time.

Phase output: a data loader/input pipeline that yields token IDs/strings, 600D FastText features, masks, NER tags when available, and severity labels when available.

10. Final neural architecture

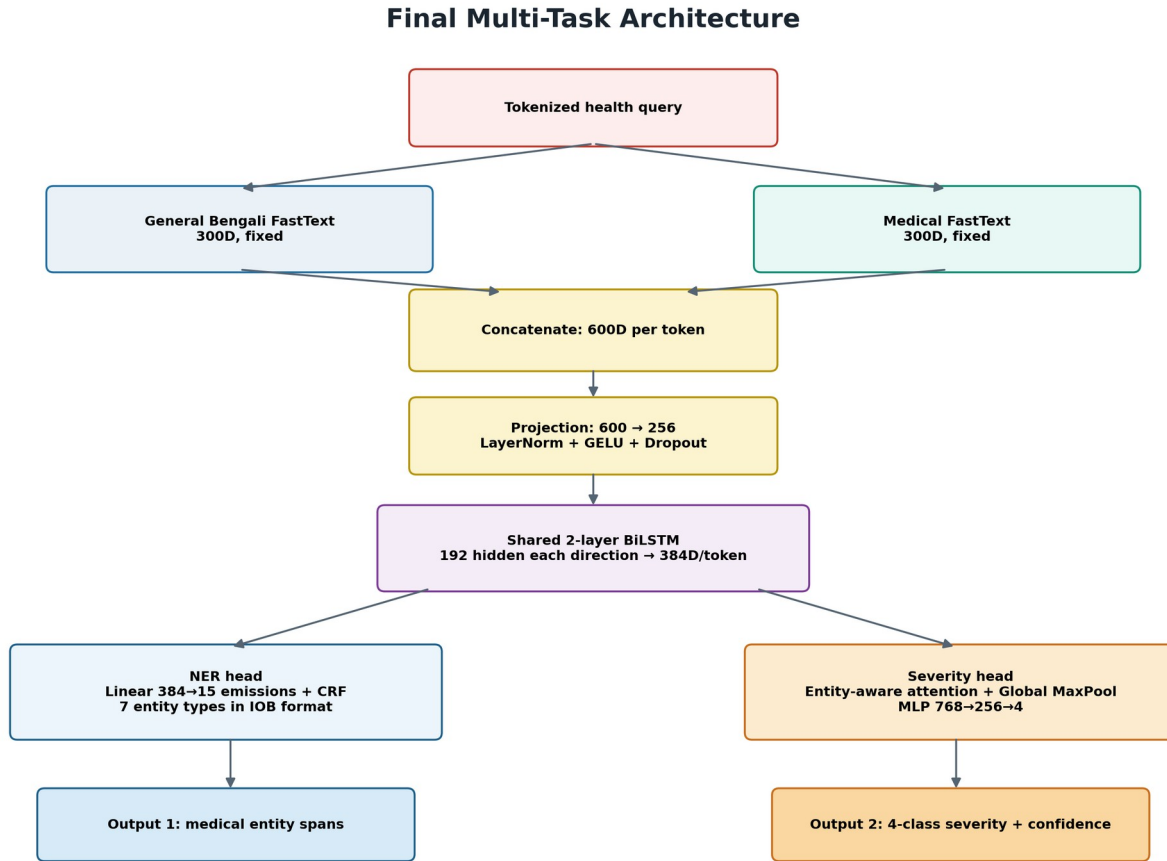


Figure 2. Fixed final architecture.

10.1 Shared projection and encoder

Block	Dimensions / setting
General FastText	300D, fixed
Medical FastText	300D, fixed
Concatenation	600D/token
Projection	600 -> 256, LayerNorm + GELU + dropout 0.20
BiLSTM	2 layers, 192 hidden units per direction
BiLSTM output	384D per token
BiLSTM dropout	0.30 between recurrent layers

The BiLSTM is shared. This is the central multi-task idea: NER training teaches the encoder where medically important concepts occur, while severity training teaches the same encoder which contexts correspond to urgency.

10.2 NER head

384D token state -> Linear(15) -> CRF

15 labels = O + B/I for 7 entity types.

The linear layer produces 15 emission scores per token. The CRF decodes the most coherent sequence of IOB labels rather than treating each token independently. This reduces invalid or implausible transitions and is a standard sequence-labeling design [6].

10.3 Entity-aware severity head

The severity head uses the shared BiLSTM states plus soft information from the NER emission layer. It does not depend on manually counted entity features.

12. Convert the 15 NER emission scores to probabilities with softmax.
13. For each entity type, add its B-type and I-type probabilities. This gives a 7D soft entity vector q_i for every token.
14. Use q_i together with the BiLSTM state h_i inside the attention scorer. Tokens that look like symptoms, medicines, age, etc. can therefore influence where severity attention focuses.

$$\text{score}_i = v^T \tanh(W_h h_i + W_q q_i + b)$$

Entity-aware attention score for token i .

$$\alpha_i = \text{softmax}(\text{score}_i), \quad h_{\text{att}} = \sum_i \alpha_i h_i$$

Attention produces one 384D sentence vector.

15. Also apply global max-pooling across the BiLSTM states. This captures the strongest learned signal anywhere in the query.

$$z = [h_{\text{att}} ; h_{\text{max}}] \rightarrow 768D$$

Attention and max-pooling are complementary.

$$768 \rightarrow \text{Dense } 256 \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.40) \rightarrow \text{Dense } 4$$

The four outputs are Emergency, Urgent, Routine, General Query.

10.4 Important gradient rule

Protect the NER head

On severity-only batches, use the NER soft probabilities as features but stop the severity loss from directly updating the NER emission/CRF head. Severity still updates the shared projection/BiLSTM through its own attention and pooling path. NER batches update the NER head normally.

This prevents the severity task from distorting the NER labeler simply because severity examples do not have gold NER tags.

10.5 Why there is no Character-CNN

FastText already models character n-grams and can produce vectors from word fragments [5]. A separate Character-CNN could help in some NER systems, but here it overlaps substantially with the chosen embedding design and adds another trainable component. The fixed final version therefore excludes it.

10.6 Why there are no entity count/presence features

The severity head already receives token-level soft NER information through entity-aware attention. Adding hard presence/count features repeats similar information and introduces brittle thresholding. The final version keeps the differentiable soft NER signal only.

11. Phase 8 - NER pretraining

Train the shared projection + BiLSTM + NER emission layer + CRF first, using HealthNER TRAIN. This gives the shared encoder a strong medical structure before it sees the much smaller severity dataset.

11.1 What is trainable

Component	NER pretraining
General FastText	Fixed
Medical FastText	Fixed
600->256 projection	Train
Shared 2-layer BiLSTM	Train
NER Linear + CRF	Train
Severity head	Not used yet

11.2 Loss

$$L_{\text{NER}} = \text{CRF negative log-likelihood}$$

Optimize the gold IOB sequence, not independent token accuracy.

11.3 Fixed starting training settings

Setting	Value
Optimizer	AdamW
Learning rate	1e-3 for projection/BiLSTM/NER head
Weight decay	1e-4
Batch size	32; reduce to 16 only if memory/long-sequence issues occur
Gradient clipping	Global norm 1.0
Maximum epochs	20
Early stopping	Patience 3 on validation entity-level F1
Mixed precision	Use on GPU
Model checkpoint	Best validation entity-F1

11.4 NER validation metrics

- Entity-level exact-span precision, recall, and F1 - primary.
- Per-entity F1 for all seven entity types.
- Micro F1 across entities.
- Token-level F1 may be reported for comparison with prior HealthNER work, but entity-level span F1 should be your practical main metric.

HealthNER is imbalanced across entity types - symptoms are much more common than some rare categories such as medical procedures [2]. Therefore overall F1 alone is insufficient; always keep the seven per-type scores.

Phase output: checkpoints/best_ner.pt + NER validation metrics.

12. Phase 9 - Severity-head warm-up

Now attach the entity-aware attention, max-pooling, and severity MLP. Before full joint training, let only the new severity head learn for a short warm-up.

12.1 Freeze/train

Component	Warm-up behavior
FastText models	Fixed
Projection	Freeze
BiLSTM	Freeze
NER emission + CRF	Freeze
Entity-aware attention	Train
Severity MLP	Train

12.2 Severity loss

$$L_{\text{severity}} = \text{standard 4-class cross-entropy}$$

No class weighting because all four classes are already around 22%-27%.

12.3 Warm-up settings

Setting	Value
Epochs	2-3
Optimizer	AdamW
Severity-head learning rate	1e-3
Batch size	32
Selection metric	Validation Macro-F1

Warm-up prevents a randomly initialized severity head from immediately sending noisy gradients into the medical encoder.

Phase output: a working severity head attached to the pretrained NER encoder.

13. Phase 10 - Joint multi-task fine-tuning

This is the final training stage. Unfreeze the shared projection and BiLSTM, then alternate NER and severity batches. Each dataset supplies only the labels it actually has.

13.1 Batch behavior

Batch type	Loss used	Parameters updated
HealthNER batch	CRF NER loss	Projection + BiLSTM + NER head
Severity batch	4-class cross-entropy	Projection + BiLSTM + severity head; NER head protected

13.2 Task schedule

Use a simple 1:1 alternating schedule: one NER batch, one severity batch, repeat. Because HealthNER is larger, the NER encoder has already seen the complete NER training set during pretraining; joint fine-tuning is primarily for maintaining NER while adapting the shared encoder to severity.

13.3 Learning rates

Parameter group	Learning rate
Shared projection + BiLSTM	2e-4

NER head on NER batches	5e-4
Severity attention + MLP	1e-3

Use AdamW, weight decay 1e-4, gradient clipping 1.0, and mixed precision. Train for at most about 10-15 joint epochs with early stopping.

13.4 Model-selection rule

Severity is the final routing objective, so choose checkpoints primarily by severity validation Macro-F1. However, do not accept a checkpoint whose NER validation F1 has collapsed. A practical guardrail is: NER validation F1 should stay within about 1 percentage point of the best NER-pretraining checkpoint unless a clearly justified trade-off is documented.

13.5 What the joint stage is trying to achieve

NER teaches “what medical information is present.” Severity teaches “how the overall context should be prioritized.” Sharing the lower encoder lets both forms of supervision shape the same contextual representation. The related idea of joint medical question classification and NER has precedent in multi-task medical NLP [7].

Phase output: checkpoints/best_joint.pt + joint validation history.

14. Phase 11 - Checkpoint selection and confidence calibration

14.1 Freeze all design decisions first

Before touching either test set, freeze preprocessing, tokenization, corpus, embedding files, architecture, hyperparameters, checkpoint-selection rule, and calibration procedure. Test sets are for the final unbiased measurement, not iterative tuning.

14.2 Temperature scaling

Take severity validation logits from the selected joint checkpoint and fit one scalar temperature T that minimizes validation negative log-likelihood. Divide final logits by T before softmax. This changes confidence calibration without changing the ranking/class prediction in the usual case.

$$P_{\text{calibrated}} = \text{softmax}(\text{logits} / T)$$

T is fitted only on severity validation data.

14.3 Low-confidence flag

If the demo displays a “low confidence - human review” warning, choose the confidence threshold from validation/calibration behavior. Do not hard-code 0.50 or 0.70 just because it looks reasonable.

Phase output: selected final checkpoint + saved temperature/calibration parameters.

15. Phase 12 - Final evaluation

Now run each test set once with the frozen model. Save predictions so later tables/figures can be regenerated without repeatedly modifying the model.

15.1 Severity test metrics

Metric	Role
Macro-F1	Primary model-quality metric; treats all four classes equally
Accuracy	Easy overall summary
Weighted-F1	Frequency-weighted summary
Per-class precision/recall/F1	Shows which classes are difficult
Emergency Recall	Safety-relevant high-priority miss rate indicator
Confusion matrix	Shows systematic class confusions
ECE / reliability diagram	Shows whether confidence is calibrated

15.2 NER test metrics

Metric	Role
Entity-level precision	Correct predicted spans / predicted spans
Entity-level recall	Correct predicted spans / gold spans
Entity-level F1	Primary span extraction metric
Per-entity F1	Seven separate entity-type results
Micro F1	Aggregate entity performance
Token F1	Optional benchmark-style comparison

15.3 Error analysis

Manually inspect a structured sample of errors. At minimum create categories for Banglish/code-switching, spelling variants, rare medical procedures/specialists, negation, long queries, multiple symptoms, medicine/dosage confusion, and severity confusions such as Emergency -> Urgent/Routine.

Error analysis is not an optional decoration. It explains why the numbers look the way they do and gives you credible future-work ideas.

15.4 Three-seed final reporting

After the pipeline is stable, repeat the complete neural training with three fixed random seeds (for example 42, 52, 62). Report mean +/- standard deviation for main validation/test metrics. Train Medical FastText only once; reuse the exact same corpus embedding across seeds. Deploy the seed whose validation result is strongest under the fixed selection rule.

Phase output: final severity results, final NER results, confusion matrix, calibration plot, and error-analysis files.

16. Required ablations and baseline

“One best version” means the final architecture is fixed. You still need a few controlled comparison runs to prove that its important components actually help.

16.1 Minimum severity baseline

Train TF-IDF + Linear SVM (or Logistic Regression) on the same severity training split. It is a fast classical reference. It does not replace the final model.

16.2 Three focused ablations

Experiment	What changes	Question answered
Full model	Nothing	Final result
Ablation 1	Remove Medical FastText; general FastText only	Does medical-domain embedding help?
Ablation 2	Remove general FastText; Medical FastText only	Does broad Bengali knowledge help?
Ablation 3	Replace entity-aware attention with ordinary attention	Does NER information improve severity?

Keep every other setting identical. Compare severity Macro-F1 and Emergency Recall. This is the cleanest way to justify the components instead of merely claiming they should work.

17. Final inference and demo

17.1 Inference sequence

16. Normalize the incoming query using the same inference preprocessing.
17. Tokenize using the same tokenizer used during training.
18. Get 300D general FastText and 300D medical FastText vector for each token.
19. Concatenate to 600D and pass through the learned 600->256 projection.

20. Run the shared 2-layer BiLSTM.
21. Decode the NER sequence with the CRF and merge B/I tokens into entity spans.
22. Compute soft entity-aware attention + max-pooling for severity.
23. Produce the four severity logits.
24. Apply the saved temperature and softmax to obtain calibrated confidence.
25. Display entity spans, severity label, confidence, and a non-diagnostic disclaimer.

17.2 Demo interface

A simple Gradio or Streamlit interface is sufficient. Keep the user experience focused: one text box, one Run button, one entity section, one severity section, confidence, and a brief safety statement.

```
Input query
↓
Extracted entities (type + text span)
+
Severity: Emergency / Urgent / Routine / General Query
Confidence: calibrated probability
+
"This system prioritizes messages; it does not diagnose or prescribe."
```

18. Project files and outputs

By the end, your project should have a structure similar to the following. Names can change, but the separation of raw data, frozen splits, embeddings, checkpoints, and results should remain.

```
banglacare/
├── data/
│   ├── raw/                                # untouched downloaded data
│   ├── splits/
│   │   ├── severity_train.*
│   │   ├── severity_val.*
│   │   ├── severity_test.*
│   │   └── healthner official train/val/test
│   └── processed/
├── corpus/
│   ├── medical_corpus.txt
│   └── corpus_stats.*
├── embeddings/
│   ├── cc.bn.300.bin
│   └── medical_fasttext.bin
├── checkpoints/
│   ├── best_ner.pt
│   ├── best_joint.pt
│   └── calibration.*
├── results/
│   ├── severity_metrics.*
│   ├── severity_predictions.*
│   ├── ner_metrics.*
│   ├── ner_predictions.*
│   ├── ablation_results.*
│   └── error_analysis.*
├── figures/
│   ├── confusion_matrix.*
│   └── reliability_plot.*
├── logs/
└── app/
```

19. Kaggle Free runtime plan

This architecture is small compared with transformer training. A Kaggle T4-class GPU is sufficient. FastText itself is primarily CPU work; the BiLSTM/CRF and joint model use the GPU.

Stage	Typical resource	Rough time after code is stable
Data audit + split + corpus	CPU/RAM	15-45 min
Medical FastText	CPU	10-30 min
Load/cache FastText features	CPU/RAM	10-30 min
NER pretraining	GPU	30-90 min
Severity warm-up	GPU	5-15 min
Joint fine-tuning	GPU	30-90 min
Calibration + final evaluation	CPU/GPU	10-20 min

A complete successful run should commonly fit in roughly 2-4 hours once the pipeline is working. Development/debugging takes longer. Three final neural seeds can usually be completed within a modest Kaggle Free GPU budget because FastText does not need to be retrained per seed.

19.1 Kaggle practical setup

- Upload the three already-downloaded datasets as a private Kaggle Dataset so they persist across notebook sessions.
- Enable Internet when you need Hugging Face BanglaHealth or the pretrained FastText download.
- Download cc.bn.300.bin once, then save it as a private Kaggle Dataset if repeated sessions make re-downloading inconvenient.
- Use one GPU; multi-GPU is unnecessary for this model.
- Save every important checkpoint/result to persistent Kaggle output or a private Kaggle Dataset before the session ends.

20. Common failure checks

Problem	Likely cause	Fix
Very high severity accuracy but poor Macro-F1	Split/label issue or class-specific failure	Inspect confusion matrix and duplicate leakage
NER tokens/labels mismatch	Preprocessing changed tokenization	Use HealthNER-provided tokenization; re-run alignment checks
NER F1 collapses during joint training	Severity gradients altered shared representation too aggressively	Lower shared LR; keep NER alternating batches; protect NER head
Entity-aware attention hurts severity	NER signal noisy or implementation wrong	Verify <code>q_i</code> construction and compare ordinary-attention ablation
Medical FastText seems useless	Corpus contamination/quality or general model already dominates	Audit nearest neighbors; verify corpus composition and no preprocessing damage
Validation excellent, test much worse	Leakage/overfitting/near duplicates	Re-check grouped split and held-out corpus decontamination
Confidence always near 1.0	Overconfidence	Use validation temperature scaling and reliability diagram
Out-of-memory	Long padded batches	Use dynamic padding, reduce batch 32->16, inspect max sequence length

21. Final report structure

Once experiments are complete, the written report should tell one clear story rather than listing disconnected models.

Section	What it should contain
Introduction	Why Bangla consumer-health query prioritization and medical entity extraction matter; define the non-diagnostic scope.
Related Work	Bangla-HealthNER, Bangla consumer-health NLP, FastText/subword embeddings, BiLSTM-CRF, multi-task medical NLU.
Datasets	Describe the four data resources, label spaces, class distribution, split policy, and leakage controls.
Corpus and Embeddings	Explain the decontaminated medical corpus, custom Medical FastText, and pretrained Bengali FastText.
Method	Explain projection, shared BiLSTM, CRF NER, entity-aware attention, max-pooling, severity MLP, and calibration.
Training	Describe NER pretraining, severity warm-up, joint alternating fine-tuning, optimizers, early stopping, and seeds.
Evaluation	NER span metrics; severity Macro-F1, Emergency Recall, confusion matrix, calibration.
Results	Main model, baseline, three focused ablations, mean +/- std.
Error Analysis	Banglish, rare entities, negation, long messages, priority confusions.
Limitations and Safety	Public/social data, annotation limits, domain shift, not a diagnosis tool, human review for uncertain cases.
Conclusion	What the custom multi-task approach achieved and what future work should test.

22. Start-to-finish checklist

- ☐ Preserve raw HealthNER, severity, and CHQ-Summ files.
- ☐ Load BanglaHealth and record its version/source.
- ☐ Download/extract cc.bn.300.bin.
- ☐ Audit all datasets and save statistics.
- ☐ Freeze HealthNER official splits.
- ☐ Create duplicate-aware stratified severity 80/10/10 splits.
- ☐ Confirm severity classes stay roughly 22%-27%; use normal CE.
- ☐ Create held-out text fingerprints.
- ☐ Apply conservative preprocessing and verify HealthNER token-label alignment.
- ☐ Build medical corpus from BanglaHealth + HealthNER train + CHQ + severity train.
- ☐ Remove exact/near held-out overlaps and corpus duplicates.
- ☐ Train 300D custom Medical FastText with the fixed skip-gram settings.
- ☐ Audit Medical FastText neighbors.
- ☐ Load general Bengali FastText and keep both embeddings fixed.
- ☐ Build 600D token features and 600->256 trainable projection.
- ☐ Build 2-layer BiLSTM (192 each direction).

- ☐ Build 15-emission NER head + CRF.
- ☐ Build entity-aware attention + max-pool + 768->256->4 severity head.
- ☐ Pretrain NER with HealthNER train; select by NER validation entity-F1.
- ☐ Warm up severity head for 2-3 epochs with encoder frozen.
- ☐ Joint fine-tune with 1:1 alternating NER/severity batches.
- ☐ Select best checkpoint by severity validation Macro-F1 with NER stability guardrail.
- ☐ Fit temperature scaling on severity validation logits.
- ☐ Freeze model and run both test sets once.
- ☐ Report NER per-entity and entity-level F1.
- ☐ Report severity Macro-F1, Accuracy, Weighted-F1, per-class metrics, Emergency Recall, confusion matrix, and calibration.
- ☐ Run TF-IDF baseline and three focused ablations.
- ☐ Repeat final neural training for three seeds and report mean +/- std.
- ☐ Build Streamlit/Gradio demo with entities + severity + calibrated confidence.
- ☐ Write final report with limitations and non-diagnostic safety boundary.

Final architecture summary

General FastText 300D + Medical FastText 300D -> 600D

Fixed embedding features.

600D -> 256D projection -> 2-layer BiLSTM -> 384D/token

Shared trainable encoder.

NER: 384D -> 15 emissions -> CRF

Seven medical entity types.

Severity: entity-aware attention(384D) + max-pool(384D) -> 768D -> 256 -> 4

Emergency / Urgent / Routine / General Query.

One-sentence project definition

A custom multi-task Bangla medical NLP model that combines general and domain-specific FastText representations with a shared BiLSTM to extract seven medical entity types and simultaneously prioritize healthcare queries into four severity classes with calibrated confidence.

References and source links

[1] Bangla Healthcare Severity Dataset (Mendeley Data, Version 1, 2026).

<https://data.mendeley.com/datasets/37z8kgx79m/1>

[2] Bangla-HealthNER: A Medical Entity Recognition Dataset for Bangla (Findings of EMNLP 2023).

<https://aclanthology.org/2023.findings-emnlp.383/>

[3] BanglaCHQ-Summ: Bangla Consumer Health Question Summarization dataset (BanglaLP 2023).

<https://aclanthology.org/2023.banglalp-1.10/>

[4] BanglaHealth: A Bengali paraphrase dataset on health domain (Data in Brief, 2025) and dataset repository.

<https://huggingface.co/datasets/faisal4590aziz/bangla-health-related-paraphrased-dataset>

[5] fastText - pretrained word vectors for 157 languages; Common Crawl + Wikipedia, 300D.

<https://fasttext.cc/docs/en/crawl-vectors.html>

[6] Neural Architectures for Named Entity Recognition (BiLSTM-CRF sequence labeling reference).

<https://aclanthology.org/N16-1030/>

[7] Medical QA Oriented Multi-Task Learning Model for Question Intent Classification and Named Entity Recognition.

<https://www.mdpi.com/2078-2489/13/12/581>

Bangla-HealthNER GitHub repository.

<https://github.com/alvi-khan/Bangla-HealthNER>

BanglaCHQ-Summ GitHub repository.

<https://github.com/alvi-khan/BanglaCHQ-Summ>

Direct official Bengali FastText binary download.

<https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.bn.300.bin.gz>

Methodology status

This document fixes the recommended custom-model methodology as of September 2026. Exact numerical performance cannot be guaranteed before experiments; the ablations are included precisely to verify that each major component contributes on your data.